

Chapter 4. Scalability and Performance

A production-ready microservice is scalable and performant. A scalable, performant microservice is one that is driven by efficiency, one that can not only handle a large number of tasks or requests at the same time, but can handle them efficiently and is prepared for tasks or requests to increase in the future. In this chapter, the essential components of microservice scalability and performance are covered, including understanding the qualitative and quantitative growth scales, hardware efficiency, identification of resource requirements and bottlenecks, capacity awareness and planning, scalable handling of traffic, the scaling of dependencies, task handling and processing, and scalable data storage.

Principles of Microservice Scalability and Performance

Efficiency is of the utmost importance in real-world, large-scale distributed systems architecture, and microservice ecosystems are no exception to this rule. It's easy to quantify the efficiency of a single system (like a monolithic application), but evaluating the efficiency and achieving greater efficiency in a large ecosystem of microservices, where tasks are sharded out between hundreds (if not thousands) of small services, is incredibly difficult. It's also bounded by the laws of computer architecture and distributed systems, which place limits on the efficiency of large-scale, complex distributed systems: the more distributed your system, and the more microservices you have in place within that system, the less of a difference the efficiency of one microservice will have on the entire system. Standardization of principles that will increase overall efficiency becomes a necessity. Two of our production-readiness standards—*scalability* and *performance*—help to achieve this overall efficiency, and increase the availability of the microservice ecosystem.

Scalability and performance are uniquely intertwined because of the effects they have on the efficiency of each microservice and the ecosystem as a whole. As we saw in [Chapter 1, *Microservices*](#), in order to build a scalable application, we need to design for concurrency and partitioning:

concurrency allows each task to be broken up into smaller pieces, while partitioning is essential for allowing these smaller pieces to be processed in parallel. So, while *scalability* is related to how we divide and conquer the processing of tasks, *performance* is the measure of how efficiently the application processes those tasks.

In a growing, thriving microservice ecosystem, where traffic is increasing steadily, each microservice needs to be able to scale with the entire system without suffering from performance problems. To ensure that our microservices are scalable and performant, we need to require several things of each microservice. We need to understand its *growth scale*, both quantitative and qualitative, so that we can prepare for expected growth. We need to use our *hardware resources efficiently*, be aware of *resource bottlenecks and requirements*, and do appropriate *capacity planning*. We need to ensure that a microservice's *dependencies will scale* with it. We need to *manage traffic* in a scalable and performant way. We need to *handle and process tasks* in a performant manner. Last but not least, we need to *store data in a scalable way*.

A PRODUCTION-READY SERVICE IS SCALABLE AND PERFORMANT

- Its qualitative and quantitative growth scales are known.
 - It uses hardware resources efficiently.
 - Its resource bottlenecks and requirements have been identified.
 - Capacity planning is automated and performed on a scheduled basis.
 - Its dependencies will scale with it.
 - It will scale with its clients.
 - Its traffic patterns are understood.
 - Traffic can be re-routed in case of failures.
 - It is written in a programming language that allows it to be scalable and performant.
 - It handles and processes tasks in a performant manner.
 - It handles and stores data in a scalable and performant way.
-

Knowing the Growth Scale

Determining *how* a microservice scales (at a very high level) is the first step toward understanding how to build and maintain a scalable microservice. There are two aspects to knowing the *growth scale* of a microservice, and they both play important roles in understanding and planning for the scalability of a service. The first is the *qualitative growth scale*, which comes from understanding where the service fits into the overall microservice ecosystem and which key high-level business metrics it will be affected by. The second is the *quantitative growth scale*, which is, as its name suggests, a well-defined, measurable, and quantitative understanding of how much traffic a microservice can handle.

The Qualitative Growth Scale

The natural tendency when trying to determine the growth scale of a microservice is to phrase the growth scale in terms of *requests per second* (RPS) or *queries per second* (QPS) that the service can support, then predicting how many RPS/QPS will be made to the service in the future. The term “requests per second” is generally used when talking about microservices, and “queries per second” when talking about databases or microservices that return data to clients, though in many cases they are interchangeable. This is very important information, but it’s useless without additional context—specifically, without the context of where the microservice fits into the overall picture.

In most cases, information about the RPS/QPS a microservice can support is determined by the state of the microservice at the time the growth scale is initially calculated: if the growth scale is calculated by only looking at the current levels of traffic and how the microservice handles the current traffic load, making any inferences about how much traffic the microservice can handle in the future runs the risk of being misguided. There are several approaches one could take to get around this problem, including load testing (testing the microservice with higher loads of traffic), which can present a more accurate picture of the scalability of the service, and analyzing historical traffic data to see how the traffic level grows over time. But there’s something very key missing here, something that is an inherent property of microservice architecture—namely, that microservices do not live alone but as part of a larger ecosystem.

This is where the *qualitative growth scale* comes in. Qualitative growth scales allow the scalability of a service to tie in with higher-level business

metrics: a microservice may, for example, scale with the number of users, with the number of people who open a phone application (“eyeballs”), or with the number of orders (for a food delivery service). These metrics, these qualitative growth scales, aren’t tied to an individual microservice but to the overall system or product(s). At the business level, the organization will have, for the most part, some idea of how these metrics will change over time. When these higher-level business metrics are communicated to engineering teams, developers can interpret them as they relate to their respective microservices: if one of their microservices is part of the order flow for a food delivery service, they will know that any metrics related to the number of orders expected in the future will tell them what kind of traffic their service should expect.

When I ask microservice development teams if they know the growth scale of their service, the usual response is, “It can handle x requests per second.” My follow-up questions are always geared toward discovering where the service in question fits into the overall product: When are requests made? Is it one request per trip? One request each time someone opens the app? One request every time a new user signs up for our product? When these context-filling questions are answered, the growth scale becomes clear—and useful. If the number of requests made to the service is directly linked to the number of people who open a phone application, then the service scales with eyeballs, and we can plan for scaling the service by predicting how many people will be opening the application. If the number of requests made to the service is determined by the number of people who order delivery food, then the service scales with deliveries, and we can plan and predict for scaling our service by using higher-level business metrics about how many future deliveries are predicted.

There are exceptions to the rules of qualitative growth scales, and determining an appropriate qualitative growth scale can become very complicated the further down the stack the service is found. Internal tools tend to suffer from these complications, and yet they tend to be so business-critical that if they aren’t scalable, the rest of the organization quickly hits scalability challenges. It’s not easy to put the growth scale of a service like a monitoring or alerting platform in terms of business metrics (users, eyeballs, etc.), so platform and/or infrastructure organizations need to determine accurate growth scales for their services in terms of their customers (developers, services, etc.) and their customers’ specifications. Internal tools can scale with, for example, number of deployments, number of ser-

vices, number of logs aggregated, or gigabytes of data. These are more complicated because of the inherent difficulty in predicting these numbers, but they must be just as straightforward and predictable as the growth scales of microservices higher in the stack.

The Quantitative Growth Scale

The second part of knowing the growth scale is determining its quantitative aspects, which is where RPS/QPS and similar metrics come into play. To determine the *quantitative growth scale*, we need to approach our microservices with the qualitative growth scale in mind: the quantitative growth scale is defined by translating the qualitative growth scale into a measurable quantity. For example, if the qualitative growth scale of our microservice is measured in “eyeballs” (e.g., how many people open a phone application), and each “eyeball” results in two requests to our microservice and one database transaction, then our quantitative growth scale is measured in terms of requests and transactions, resulting in requests per second and transactions per second as the two key quantities determining our scalability.

The importance of choosing accurate qualitative and quantitative growth scales cannot be overemphasized. As we will soon see, the growth scale will be used when making predictions about the service’s operational costs, hardware needs, and limitations.

Efficient Use of Resources

When considering the scalability of large-scale distributed systems like microservice ecosystems, one of the most useful abstractions we can make is to treat properties of our hardware and infrastructure systems as *resources*. CPU, memory, data storage, and the network are similar to resources in the natural world: they are finite, they are physical objects in the real world, and they must be distributed and shared between various key players in the ecosystem. As we discussed briefly in [“Organizational Challenges”](#), hardware resources are expensive, valuable, and sometimes rare, which leads to fierce competition for resources within the microservice ecosystem.

The organizational challenge of resource allocation and distribution can be alleviated by giving business-critical microservices a greater share of the resources. Resource needs can be prioritized by categorizing various microservices within the ecosystem according to their importance and value to the overall business: if resources are scarce across the ecosystem, the most business-critical services can be given higher priority with regard to resource allocation.

The technical challenge of resource allocation and distribution presents some difficulty, because many decisions need to be made about the first layer (the hardware layer) of the microservice ecosystem. Microservices can be given dedicated hardware so that only one service will run on each host, but this can be rather expensive and an inefficient use of hardware resources. Many engineering organizations opt to share hardware among multiple microservices, and each host will run several different services—a practice that is, in most cases, a more efficient use of hardware resources.

THE DANGERS OF SHARED HARDWARE RESOURCES

While running many different microservices on one machine (that is, sharing machines between microservices) is usually a more efficient use of hardware resources, care must be taken to ensure that the microservices are sufficiently isolated and don't compromise the performance, efficiency, or availability of their neighboring microservices. Containerization (using Docker) along with resource isolation can help prevent microservices from being harmed by badly behaved neighbors.

One of the most effective ways to allocate and distribute hardware resources across a microservice ecosystem is to fully abstract away the notion of a host and replace it with hardware resources using resource abstraction technologies like Apache Mesos. Using this level of resource abstraction allows resources to be allocated dynamically, eliminating many of the pitfalls associated with resource allocation and distribution in large-scale distributed systems like microservice ecosystems.

Resource Awareness

Before hardware resources can be efficiently allocated and distributed to microservices within the microservice ecosystem, it is important to identify the *resource requirements* and *resource bottlenecks* of each microservice. Resource requirements are the specific resources (CPU, RAM, etc.) that each microservice needs; identifying these is essential for running a scalable service. Resource bottlenecks are the scalability and performance limitations of each individual microservice that are dependent on features of its resources.

Resource Requirements

The *resource requirements* of a microservice are the hardware resources the microservice needs in order to run properly, to process tasks efficiently, and to be scaled vertically and/or horizontally. The two most important and relevant hardware resources tend to be, unsurprisingly, CPU and RAM (in multithreaded environments, threads become the third important resource). Determining the resource requirements of a microservice then entails quantifying the CPU and RAM that *one instance* of the service needs in order to run. This is essential for resource abstraction, for resource allocation and distribution, and for determining the overall scalability and performance of the microservice.

IDENTIFYING ADDITIONAL RESOURCE REQUIREMENTS

While CPU and RAM are the two most common resource requirements, it's important to keep an eye out for other resources that a microservice may need within the ecosystem. These can be hardware resources like database connections or application platform resources like logging quotas. Being aware of the needs of a specific microservice can do a lot to improve scalability and performance.

Calculating the specific resource requirements of a microservice can be a tricky, lengthy process, because there are many relevant factors. The key here, as I mentioned earlier, is to determine what the requirements are for only *one instance* of the service. The most effective and efficient way to scale our service is to scale it horizontally: if our traffic is about to increase, we want to add a few more hosts and deploy our service to those new hosts. In order for us to know how many hosts we need to add, we need to know what our service looks like running on only one host: how much traffic can it handle? how much CPU does it utilize? how much

memory? Those numbers will tell us exactly what the resource requirements of our microservice are.

Resource Bottlenecks

We can discover and quantify the performance and scalability limitations of our microservices by identifying *resource bottlenecks*. A resource bottleneck is anything inherent about the way the microservice utilizes its resources that limits the scalability of the application. This could be an infrastructure bottleneck or something within the architecture of the service that prevents it from being scalable. For example, the number of open database connections a microservice needs can be a bottleneck if it nears the connection limit of the database. Another example of a common resource bottleneck is when microservices need to be vertically scaled (rather than horizontally scaled, where more instances/hardware is added) when they experience an increase in traffic: if the only way to scale a microservice is to increase the resources of each instance (more CPU, more memory), then the two principles of scalability (concurrency and partitioning) are abandoned.

Some resource bottlenecks are easy to identify. If your microservice can only be scaled to meet growing traffic by deploying it to machines with more CPU and memory, then you have a scalability bottleneck and need to refactor the microservice so that it can be scaled horizontally rather than vertically, using concurrency and partitioning as your guiding principles.

THE PITFALLS OF VERTICAL SCALING

Vertical scaling isn't a sustainable or scalable way to architect microservices. It may appear to work out all right in situations where each microservice has dedicated hardware, but it will not work well with the new hardware abstraction and isolation technologies that are used in the tech world today, like Docker and Apache Mesos. Always optimize for concurrency and partitioning if you want to build a scalable application.

Other resource bottlenecks are not as obvious, and the best way to discover them is to run extensive load testing on the service. We will cover load testing in much greater detail in [“Resiliency Testing”](#).

Capacity Planning

One of the most important requirements of building a scalable microservice is ensuring that it will have access to necessary and required hardware resources as it scales. Efficiently using resources, planning for growth, and designing a microservice for perfect efficiency and scalability from the ground up is all quickly made useless if no hardware resources are available when the microservice needs to host more production traffic. This challenge is especially relevant for microservices that are optimized for horizontal scalability.

In addition to the technical challenges that accompany this potential problem, engineering organizations are often faced with larger organizational-level and business-relevant issues that come along for the ride: hardware resources cost quite a bit of money, businesses and individual development teams within them have budgets to adhere to, and these budgets (which tend to include hardware) need to be planned for in advance. To ensure that microservices can scale properly when traffic increases, we can perform scheduled *capacity planning*. The principles of capacity planning are pretty straightforward: determine the hardware needs of each microservice in advance, build the needs into the budget, and make sure that the required hardware is reserved.

To determine the hardware needs of each service, we can use the growth scales (both quantitative and qualitative), key business metrics and traffic predictions, the known resource bottlenecks and requirements, and historical data about the microservice's traffic. This is where qualitative and quantitative growth scales come in especially handy, because they allow us to figure out precisely how the scalability behavior of our microservices relate to high-level business predictions. For example, if we know that (1) our microservice scales with unique visitors to the overall product, (2) each unique visitor corresponds to a certain number of requests per second made to our microservice, and (3) that the company predicts that the product will receive 20,000 new unique visitors in the next quarter, then we'll know exactly what our capacity needs will be for the next quarter.

This needs to be built into the budget of each development team, each engineering organization, and each company. Running this exercise on a

scheduled basis *before* budgeting is determined can help engineering organizations make sure that hardware resources are never unavailable simply because resource budgeting wasn't completed or prepared for. The important thing here (from both the engineering and business perspectives) is to recognize the cost of inadequate capacity planning: microservices that can't scale properly because of hardware shortages lead to decreased availability within the entire ecosystem, which leads to outages, which costs the company money.

LEAD TIME FOR NEW HARDWARE REQUESTS

One potential problem that's commonly overlooked by development teams during the capacity planning phase is that the hardware that is needed for the microservice might not exist at the time of planning and may need to be acquired, installed, and configured before any microservices can run on it. Before scheduling capacity planning, take care to find out the exact lead time needed for acquiring new hardware in order to avoid long shortages in critical times, and allow some room for delays in the process.

Once the hardware resources have been secured and dedicated to each microservice, capacity planning is complete. Determining when and how to allocate the hardware after the planning phase is, of course, up to each engineering organization and their development, infrastructure, and operations teams.

Capacity planning can be a really difficult and manual task. Like most manual tasks within engineering, it introduces new modes of failure: manual calculations can be off, and even a small shortage can prove disastrous to business-critical services. Automating the majority of the capacity planning process away from development and operations teams cuts down on potential errors and failures, and a great way to accomplish this is to build and run a capacity planning self-service tool within the application platform layer of the microservice ecosystem.

Dependency Scaling

The scalability of a microservice's dependencies can present a scalability problem of its own. A microservice that is architected, built, and run to be

perfectly scalable in every way still faces scalability challenges if its dependencies cannot scale with it. If even one critical dependency is unable to scale with its clients, then the entire dependency chain suffers. Ensuring that all dependencies will scale with a microservice's expected growth is essential for building production-ready services.

This challenge is relevant to every individual microservice and every part of the microservice ecosystem stack, which means that microservice teams also need to make sure that their service isn't a scalability bottleneck for its clients. In other words, additional complexity is introduced by the rest of the microservice ecosystem. The inevitable additional traffic and growth from a microservice's clients need to be prepared for.

QUALITATIVE GROWTH SCALES AND DEPENDENCY SCALABILITY

When dealing with incredibly complex dependency chains, making sure that all microservice teams tie the scalability of their services to high-level business metrics (using the qualitative growth scale) can make sure that all services are properly prepared for expected growth, even when cross-team communication becomes difficult.

The problem of dependency scaling is an especially strong argument for the implementation of scalability and performance standards across every part of the microservice ecosystem. Most microservices do not live in isolation. Nearly every single microservice is a small part of large, intertwined, intricate dependency chains. In most cases, scaling the entire overall product, the organization, and the ecosystem effectively requires that each piece of the system scales together with the rest. Having a small number of super efficient, performant, and scalable microservices in a system where the rest of the services aren't held to (and don't meet) the same standards renders the efficiency of the standardized services completely moot.

Aside from standardization across the ecosystem, and holding each microservice development team to high scalability standards, it's important that development teams work together across microservice boundaries to ensure that each dependency chain will scale together. The development teams responsible for any dependencies of a microservice need to be alerted when increases in traffic are expected. Cross-team communica-

tion and collaboration are essential here: regularly communicating with clients and dependencies about a service's scalability requirements, status, and any bottlenecks can help to guarantee that any services that rely on each other are prepared for growth and aware of any potential scalability bottlenecks. A strategy that I've used to help teams accomplish this is by holding architecture and scalability overview meetings with teams whose services rely on one another. In these meetings, we cover the architecture of each service and its scalability limitations, then discuss together what needs to be done to scale the entire set of services.

Traffic Management

As services scale, and the number of requests each service must handle grows, a scalable, performant service must also handle traffic intelligently. There are several aspects to managing traffic in a scalable, performant way: first of all, the growth scale (quantitative and qualitative) needs to be used to predict future increases (or decreases) in traffic; second, the traffic patterns must be well understood and prepared for; and third, microservices need to be able to intelligently handle increases in traffic, as well as surges or bursts of traffic.

We've already covered the first aspect earlier in this chapter: understanding the growth scales (both quantitative and qualitative) of a microservice allows us to understand current traffic loads on the service as well as prepare for future traffic growth.

Understanding current traffic patterns helps when interacting with the service on the ground floor in a lot of really interesting ways. When traffic patterns are clearly identified, both in terms of the requests per second sent to the service over time and all key metrics (see [Chapter 6, *Monitoring*](#), for more about key metrics), changes to the service, operational downtimes, and deployments can be scheduled to avoid peak traffic times, cutting down on possible future outages if a bug is deployed and on potential downtime if the microservice is restarted while experiencing peak traffic load. Closely monitoring the traffic in light of the traffic patterns and tuning the monitoring thresholds carefully with the traffic patterns of the microservice in mind can help catch any issues and incidents quickly before they cause an outage or lead to decreased availability (the

principles of production-ready monitoring are covered in greater detail in [Chapter 6, *Monitoring*](#)).

When we can predict future traffic growth and understand the current and past traffic patterns well enough to know how the patterns will change with expected growth, we can perform load testing on our services to make sure that they behave as we expect under heavier traffic loads. The details of load testing are covered in [“Resiliency Testing”](#).

The third aspect of traffic management is where things get especially tricky. The way a microservice handles traffic should be scalable, which means it should be prepared for drastic changes in traffic, especially bursts of traffic, handle them carefully, and prevent them from taking down the service entirely. It’s easier said than done, because even the most well-monitored, scalable, and performant microservices can experience monitoring, logging, and other general issues if traffic suddenly spikes. These sorts of spikes should be prepared for at the infrastructure level, within all monitoring and logging systems, and by the development team as part of the service’s resiliency testing suite.

There’s one additional aspect I want to mention that’s related to management of traffic between and across various locations. Many microservice ecosystems won’t be deployed only in one location, one datacenter, or one city, but rather across multiple datacenters across the country (or even the world). It’s not uncommon for datacenters themselves to experience large-scale outages, and when this happens, the entire microservice ecosystem can (and usually will) go down with the datacenter. Distributing and routing traffic appropriately between datacenters is the responsibility of the infrastructure level (in particular, the communication layer) of the microservice ecosystem, but each microservice needs to be prepared to re-route traffic from one datacenter to another without the service experiencing any decreased availability.

Task Handling and Processing

Every microservice in the microservice ecosystem will need to process tasks of some sort. That is, every microservice will be receiving requests from upstream client services who either need some sort of information from the microservice or need the microservice to compute or process

something and then return information about that computation or process, and then the microservice will need to fulfill that request (usually by communicating with downstream services in addition to doing some work of its own) and return any requested information or response to the client that sent the request.

Programming Language Limitations

Microservices can accomplish this and play their required role in a myriad of ways, and the ways in which they will perform computations, interact with downstream services, and process various tasks will depend on the language that the service is written in, and consequently, on the architecture of the service (which is, in many ways, determined by the language). For example, a microservice written in Python has a number of ways that it can process various tasks, some of which require the use of asynchronous frameworks (like Tornado) and others which can utilize messaging technologies like RabbitMQ and Celery to efficiently process tasks. For these reasons, a microservice's ability to handle and process tasks in a scalable and performant manner is dictated in part by choice of language.

BEWARE OF SCALABILITY AND PERFORMANCE LIMITATIONS OF PROGRAMMING LANGUAGES

Many programming languages are not optimized for the performance and scalability requirements of microservice architecture, or do not have scalable or performant frameworks that allow microservices to process tasks efficiently.

Because of the limitations introduced by language choice when it comes to a microservice's ability to process tasks efficiently, language choice becomes extremely important in microservice architecture. To many developers, one of the selling points of the adoption of microservice architecture is the ability to write a microservice in any language, and this is usually true, but with a caveat: programming language constraints need to be taken into account, and language choice should be determined not by whether a language is fashionable or fun (or even whether it is the most common language that the development team is familiar with), but with the performance and scalability limitations of each potential language held as the deciding factors. There is no one "best" language to write a mi-

crosservice in, but there *are* languages that are better suited than others to certain types of microservices.

Handling Requests and Processing Tasks Efficiently

Language choice aside, production-readiness standardization requires each microservice to be both scalable and performant, which means that microservices need to be able to handle and process a large number of tasks at the same time, handle and process those tasks efficiently, and be prepared for tasks and requests to increase in the future. With this in mind, development teams should be able to answer three basic questions about their microservices: how their microservice processes tasks, how efficiently their microservice processes those tasks, and how their microservice will perform as the number of requests scales.

To ensure scalability and performance, microservices need to process tasks efficiently. In order to do this, they need to have both concurrency and partitioning. Concurrency requires that the service can't have one single process that does all of the work: that process will pick up one task at a time, complete the steps in a specific order, and then move on to the next, which is a relatively inefficient way to process tasks. Instead of architecting our service to use a single process, we can introduce concurrency so that each task is broken up into smaller pieces.

WRITE MICROSERVICES IN PROGRAMMING LANGUAGES THAT ARE OPTIMIZED FOR CONCURRENCY AND PARTITIONING

Some languages are better suited for efficient (concurrent and partitioned) task handling and processing than others. When writing a new microservice, make sure that the language the service is being written in won't introduce scalability and performance constraints on the microservices. Microservices that are already written in languages with efficiency limitations can (and should) be rewritten in more appropriate languages, a time consuming but incredibly rewarding task that can drastically improve scalability and performance. For example, if you are optimizing for concurrency and partitioning, and want to use an asynchronous framework to help you accomplish that, writing your service in Python (rather than C++, Java, or Go—three languages built for concurrency and partitioning) is going to introduce a lot of scalability and performance bottlenecks that will be difficult to mitigate.

Taking the smaller pieces of these tasks, we can process them more efficiently using partitioning, where each task is not only broken up into small pieces but can be processed in parallel. If we have a large number of small tasks, we can process them all at the same time by sending them to a set of workers that can process them in parallel. If we need to process more tasks, we can easily scale with the increased demand by adding additional workers to process the new tasks without affecting the efficiency of our system. Together, concurrency and partitioning help ensure that our microservice is optimized for both scalability and partitioning.

Scalable Data Storage

Microservices need to *handle data in a scalable and performant way*. The way in which a microservice stores and handles data can easily become the most significant limitation or constraint that keeps it from becoming scalable and performant: choosing the wrong database, the wrong schema, or a database that doesn't support test tenancy can end up compromising the overall availability of a microservice. Choosing the right database for a microservice is a topic that, like all the other topics covered in this book, is incredibly complex, and we will only scratch the surface in this chapter. In the following sections, we'll take a look at several things to consider when choosing databases in microservice ecosystems, and then at some database challenges that are specific to microservice architecture.

Database Choice in Microservice Ecosystems

Building, running, and maintaining databases in large microservice ecosystems is not an easy task. Some companies adopting microservice architecture opt to allow development teams to choose, build, and maintain their own databases, while others will decide on at least one database option that works for the majority of the microservices at the company, and build a separate team to run and maintain the database(s) so that developers can focus solely on their own microservices.

If we think about microservice architecture as being composed of four separate layers (see [“Microservice Architecture”](#) for more details) and recognize that, thanks to the Inverse Conway's Law, the engineering organizations of companies that adopt microservice architecture will mirror

the architecture of its product, then we can see where the responsibility for choosing the appropriate databases, building them, running them, and maintaining them lies: either in the application platform layer, which would allow databases to be provided as a service to microservice teams, or the microservice layer, where the database used by a microservice is considered part of the service. I've seen both of these setups in practice at various companies, and some work better than others. I've also noticed that one approach to this works particularly well: offering databases as a service within the application platform layer, and then allowing individual microservice development teams to run their own database if the databases offered as part of the application platform do not fit their specific needs.

The most common types of databases are *relational databases* (SQL, MySQL) and *NoSQL databases* (Cassandra, Vertica, MongoDB, and key-value stores like Dynamo, Redis, and Riak). Choosing between a relational database and a NoSQL database, and then choosing the specific appropriate database for a microservice's needs depends on the answers to several questions:

- What are the needed transactions per second of each microservice?
- What type of data does each microservice need to store?
- What is the schema needed by each microservice? And how often will it need to be changed?
- Do the microservices need strong consistency or eventual consistency?
- Are the microservices read-heavy, write-heavy, or both?
- Does the database need to be scaled horizontally or vertically?

Regardless of whether the database is maintained as part of the application platform or by each individual microservice development team, database choice should be driven by the answers to those questions. For example, if the database in question needs to be scaled horizontally, or if reads and writes need to be made in parallel, then a NoSQL database should be chosen, since relational databases struggle with horizontal scaling and parallel reads and writes.

Database Challenges in Microservice Architecture

There are several challenges with databases that are specific to microservice architecture. When databases are shared among microservices, competition for resources kicks in, and some microservices may utilize more than their fair share of the available storage. Engineers building and maintaining shared databases need to design their data storage solutions so that the database can be easily scaled if any of the tenant microservices either require additional space or are running the risk of using up all available space.

WATCH OUT FOR DATABASE CONNECTIONS

Some databases have strict limitations on the number of database connections that can be open simultaneously. Make sure that all connections are closed appropriately to avoid compromising both a service's availability and the availability of the database to all microservices that use it.

Another challenge microservices often face, especially once they've built and standardized stable and reliable development cycles and deployment pipelines, is the handling of test data from end-to-end testing, load testing, and any test writes done in staging. As mentioned in [“The Deployment Pipeline”](#), the staging phase of the deployment pipeline requires reading and/or writing to databases. If full staging has been implemented, then the staging phase will have its own separate test and staging database, but partial staging requires read and write access to production servers, so great care needs to be taken to ensure that test data is handled appropriately: it needs to be clearly marked as test data (a process known as *test tenancy*), and then all test data must be deleted at regular intervals.

Evaluate Your Microservice

Now that you have a better understanding of scalability and performance, use the following list of questions to assess the production-readiness of your microservice(s) and microservice ecosystem. The questions are organized by topic, and correspond to the sections within this chapter.

Knowing the Growth Scale

- What is this microservice's qualitative growth scale?
- What is this microservice's quantitative growth scale?

Efficient Use of Resources

- Is the microservice running on dedicated or shared hardware?
- Are any resource abstraction and allocation technologies being used?

Resource Awareness

- What are the microservice's resource requirements (CPU, RAM, etc.)?
- How much traffic can one instance of the microservice handle?
- How much CPU does one instance of the microservice require?
- How much memory does one instance of the microservice require?
- Are there any other resource requirements that are specific to this microservice?
- What are the resource bottlenecks of this microservice?
- Does this microservice need to be scaled vertically, horizontally, or both?

Capacity Planning

- Is capacity planning performed on a scheduled basis?
- What is the lead time for new hardware?
- How often are hardware requests made?
- Are any microservices given priority when hardware requests are made?
- Is capacity planning automated, or is it manual?

Dependency Scaling

- What are this microservice's dependencies?
- Are the dependencies scalable and performant?
- Will the dependencies scale with this microservice's expected growth?
- Are dependency owners prepared for this microservice's expected growth?

Traffic Management

- Are the microservice's traffic patterns well understood?
- Are changes to the service scheduled around traffic patterns?
- Are drastic changes in traffic patterns (especially bursts of traffic) handled carefully and appropriately?
- Can traffic be automatically routed to other datacenters in case of failure?

Task Handling and Processing

- Is the microservice written in a programming language that will allow the service to be scalable and performant?
- Are there any scalability or performance limitations in the way the microservice handles requests?
- Are there any scalability or performance limitations in the way the microservice processes tasks?
- Do developers on the microservice team understand how their service processes tasks, how efficiently it processes those tasks, and how the service will perform as the number of tasks and requests increases?

Scalable Data Storage

- Does this microservice handle data in a scalable and performant way?
- What type of data does this microservice need to store?
- What is the schema needed for its data?
- How many transactions are needed and/or made per second?
- Does this microservice need higher read or write performance?
- Is it read-heavy, write-heavy, or both?
- Is this service's database scaled horizontally or vertically? Is it replicated or partitioned?
- Is this microservice using a dedicated or shared database?
- How does the service handle and/or store test data?